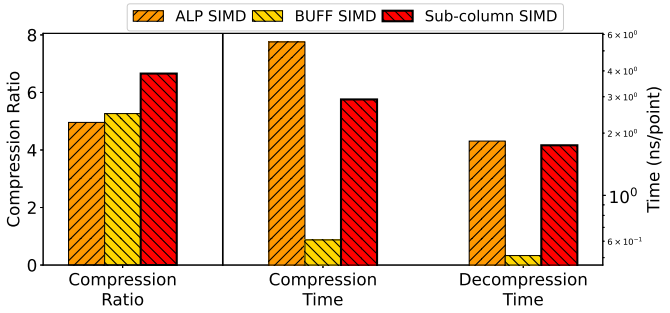


Fig. 15. Compression and decompression time of SIMD implementations over all datasets.

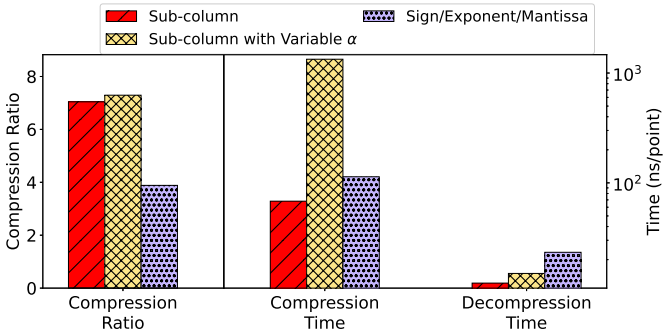


Fig. 16. Compression performance of fixed-width and variable-width sub-columns, sign/exponent/mantissa split, and Huffman encoding.

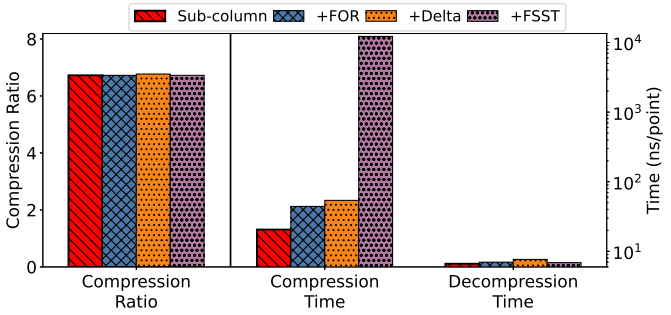


Fig. 17. Sub-column compression with additional encoding candidates.

APPENDIX

A. SIMD Optimization

Operating at a sub-byte level may be expensive and inefficient due to under-utilization of wide registers on modern architectures. To mitigate this, we describe an efficient SIMD implementation of Algorithm 1 on x86 processors using AVX2 instructions. Rather than processing each sub-byte individually, we can pack the sub-bytes of multiple values in the same sub-column together, and conduct vectorized RLE, DE or BPE compression. In this way, the wide registers could be more effectively utilized.

SIMD implementation can be applied to bit extraction for candidate sub-columns in Line 3 of Algorithm 1. Our implementation uses an AVX2 bit-matrix transpose to compute bit-plane and transition statistics for all bit columns once per block. For BPE, vectorized shifts, masks and fixed-width packing kernels extract and encode multiple sub-column values in parallel. For decompression, unpacking, dictionary lookup and RLE reconstruction are combined with sub-column restoration to reconstruct multiple values together.

We implement this version in C++17 and compare it with the original ALP implementation and a local C++ AVX2 implementation of BUFF's byte-sliced layout. The experiments use the same scaled integer sequence for all three methods; ALP receives exact double representations of these integers. We use 1024-value blocks on an AMD EPYC 7542 processor and compile with Clang 22. Normal parameter selection and memory allocation are timed, while input conversion and correctness checks are outside timing. At the same block size, our scalar reference and SIMD implementation emit identical compressed bytes and restore identical values, preserving the compression ratio.

Figure 15 reports average compression and decompression time over the 14 datasets. Sub-column compression requires 2.91 ns per point for compression and 1.75 ns per point for decompression, compared with 5.52 and 1.83 ns per point for ALP, respectively. Thus, our SIMD implementation achieves about $1.90\times$ faster compression than ALP with comparable decompression time. BUFF requires 0.61 and 0.51 ns per point, respectively.

B. Variable-length Sub-columns

Determining the optimal non-uniform sub-column widths may further reduce the sub-column storage size, but introduces extra cost to record the distinct width for each sub-column. The overall improvement of compression might be marginal. On the other hand, non-uniform sub-column widths introduce additional computation, with 2^{31} possible bit-width combinations for 32-bit values. We perform further experiments on variable-length sub-columns using the 13 datasets shared by the original variants. Figure 16 shows that non-uniform sub-column widths achieve only marginal improvements of compression ratio, while increasing compression time substantially and decompression time more moderately.

By exploiting the structure inside the values such as sign/exponent/mantissa in floats, a small fluctuation of values may lead to very different representations in sub-columns. For example, the binary mantissas of 3479.186 and 3446.915 are distinct. In contrast, by converting floats to integers with scaling [19], the more significant sub-columns of two values are the same and effective in compression. As shown in Figure 16, this split gives a much worse compression ratio.

C. Additional Encoding Candidates

We further evaluate whether additional encoding methods improve sub-column compression. Starting from BPE, RLE and DE, we separately add frame-of-reference (FOR), delta or FSST as a candidate for each sub-column. FOR subtracts the minimum within the sub-column and bit-packs the residuals.

Delta encodes differences between consecutive sub-column values. FSST treats the raw symbol bytes of each sub-column as a binary string and trains a separate dictionary.

The average compression ratio increases from 6.7246 to 6.7703 with delta, corresponding to an improvement of 0.68%. FOR and FSST participate in candidate selection but are not selected on these datasets, yielding the same compression ratio as the original method.

Compression time includes candidate evaluation and the common planning overhead of the extended selection implementation. For FSST, it also includes training and exporting a dictionary for every attempted sub-column, even when FSST is not selected. This per-sub-column training incurs substantial compression time for the short byte strings in this experiment. The results therefore concern this integration of FSST with independently trained sub-columns.